

1. Technologie-Stack

Frontend

- Next.js 14 (App Router, Standalone-Build)
- React 18
- Tailwind CSS + shadcn/ui
- NextAuth.js v4 (Session-Verwaltung)
- TanStack Query + SWR (Datenabruf)
- Framer Motion, Lucide Icons, ReactMarkdown

Backend

- C# / ASP.NET Core 8 (.NET 8)
- Microsoft.AspNetCore.OData 8 (REST + OData)
- Entity Framework Core 8 (Npgsql / PostgreSQL)
- JWT Bearer Authentication
- BCrypt.Net (Passwort-Hashing)
- Serilog (strukturiertes Logging)

Datenhaltung & Integrationen

- PostgreSQL 16 (Primärdatenbank)
- Redis 7 (Caching / Health-Check)
- MinIO / S3 (AWSSDK.S3 — Datei-Uploads)
- MailKit / SMTP (Microsoft Exchange, E-Mail)

DevOps / Deployment

- Docker & Docker Compose (Ubuntu Server)
- Nginx als Reverse Proxy (SSL/TLS)
- Multi-Stage Docker-Builds (SDK → Runtime)
- Health Checks (PostgreSQL, Redis)

Hinweis: Das Frontend (Next.js) konsumiert ausschliesslich die C#-Backend-API. Die frühere Umsetzung auf Basis von Next.js API-Routes und Prisma ORM wurde vollständig durch das ASP.NET-Core-/OData-Backend abgelöst (siehe Context-Versionen, V.2.0).

2. Architektur

Die Applikation ist als Client-Server-Architektur mit klarer Schichtentrennung aufgebaut:

- **Presentation Layer** — Next.js 14 App Router (React Client Components), Session über NextAuth.js
- **Reverse Proxy** — Nginx terminiert SSL/TLS und leitet Anfragen an Frontend (``frontend:3000``) bzw. Backend (``api:8080``)
- **API Layer** — ASP.NET Core Controller: OData-Controller (Abfrage & CRUD) unter ``/odata/*`` und klassische REST-Controller unter ``/api/*``
- **Business Logic** — Services (AuthService, RbacService, KpiService, OkrService, IcsService, NotificationService, S3StorageService ...)

- **Cross-Cutting** — SecurityHeadersMiddleware, AuditMiddleware, Rate-Limiting, JWT-Authentifizierung, RBAC-Autorisierung
- **Data Access** — Entity Framework Core (`BandSpiritDbContext`) auf PostgreSQL; automatisches Audit über die `SaveChangesAsync`-Override

3. Datenmodell

Das System umfasst 27 Entity-Framework-Core-Entitäten. Alle fachlichen Entitäten erben von `AuditableEntity` (Id, CreatedAt/By, LastModifiedAt/By). Auszug der zentralen Entitäten:

Entität	Beschreibung	Relationen
User	Systembenutzer	PasswordResetTokens, Rollen-Zuweisungen
S3Circle	Soziokratischer Kreis (S3)	S3Role, Links, Reviews, Meetings, Proposals
S3Role	Kreisrolle	S3Circle, S3PersonRoleAssignment
S3PersonRoleAssignment	Rollenzuweisung Person↔Rolle	User, S3Role
S3Meeting / ...AgendaItem	Meetings & Agenda	S3Circle, Agendaltems (1:N)
S3Proposal / Objection / Decision	Consent-Entscheidungsprozess	S3Circle, User
S3Driver / SpannungWorkItem	Treiber & Spannungen	User (Reporter/Assignee)
OKR / KeyResult	Objectives & Key Results	KeyResults (1:N)
KpiDefinition / KpiMeasurement	KPI-Definition & Messwerte	Messwerte (1:N)
SupportTicket / FAQ	Hilfe & Support	User (Reporter/Assignee)
BIKompassVersion / BIGuideNews	BI-Kompass & BI-Guide	Ersteller (User)
RolePermission	DB-basierte RBAC	Rolle + Permission
Firma	Firmeneinstellungen	OData-Singleton, Logo via S3/MinIO

Entität	Beschreibung	Relationen
Stammdaten / AppLog	Konfiguration & Logs	Schlüssel/Wert, Serilog-Sink

Das vollständige Datenmodell inkl. aller Felder und Enums findet sich unter [ERD / Datenmodell](#).

4. Sicherheit

- **Authentifizierung:** JWT Bearer Tokens (Gültigkeit 8 Stunden), ausgestellt vom `JwtTokenService`. Frontend hält die Session über `NextAuth.js` (Credentials-Provider ruft `/api/auth/login` auf).
- **Password-Hashing:** `BCrypt.Net` (`BCrypt.Net.BCrypt.HashPassword/Verify`, Standard-WorkFactor 11). Passwort-Reset per Token (1 Stunde gültig).
- **Autorisierung:** RBAC über `PermissionAuthorizationHandler` und die `PermissionRegistry.Permissions.All`; Berechtigungen werden vom `RbacService` ermittelt (60 s In-Memory-Cache mit DB-Fallback).
- **Single Sign-On:** Microsoft Entra ID (Azure AD) via `/api/auth/entra-callback` — Zuordnung über die E-Mail-Adresse zu bestehenden, aktiven Konten.
- **Rate-Limiting:** Fixed-Window-Limiter — «auth» 5 Anfragen/60 s, «api» 100 Anfragen/10 s.
- **Security-Header:** `SecurityHeadersMiddleware` setzt u.a. X-Content-Type-Options, Referrer-Policy, Permissions-Policy, HSTS.
- **Audit-Trail:** `AuditMiddleware` + automatische Audit-Felder (`CreatedBy/LastModifiedBy`) in `BandSpiritDbContext.SaveChangesAsync`.
- **Datenzugriff:** Entity Framework Core mit parametrisierten Queries (kein Raw-SQL) — Schutz vor SQL-Injection.
- **Fehlerbehandlung:** Keine internen Fehlermeldungen in API-Responses, generische deutschsprachige Texte.

5. E-Mail-Benachrichtigungen

- **NotificationService:** E-Mail-Versand über MailKit/SMTP (Microsoft Exchange / Office 365, StartTLS oder SSL).
- **Password-Reset:** Token-basierter Reset-Link per E-Mail.
- **Support-Ticket:** Benachrichtigung an die Administration bei neuen Tickets.
- **Meeting-Einladung:** Versand inkl. ICS-Kalendereintrag (`IcsService`).

Ist kein SMTP konfiguriert, wird der Versand protokolliert und übersprungen (kein Abbruch).

6. Deployment

- **Orchestrierung:** Docker Compose mit den Services `nginx`, `frontend`, `api`, `postgres` (16), `redis` (7) und `minio`.
- **Reverse Proxy:** Nginx (1.27) leitet `/odata` und `/api` an das Backend, alles Übrige an das Frontend.
- **Backend-Image:** Multi-Stage Dockerfile (SDK 8.0 → aspnet 8.0 Runtime), Non-Root-User, Health-Check via `curl`.
- **Datenbank:** PostgreSQL 16 mit persistentem Volume; EF-Core-Migrationen werden beim Start angewendet und Seed-Daten angelegt (DataSeeder).
- **Objektspeicher:** MinIO (S3-kompatibel) für Logo- und Datei-Uploads.
- **Betrieb:** Ubuntu Server; SSL/TLS über Nginx. Details siehe [Installationsanleitung](#).